



performance Testing

The Complete Software Testing Bootcamp

Agenda

- 1 **What is Performance Testing**
- 2 **Types of Performance Testing**
- 3 **Load Generation**
- 4 **Load Profiles**
- 5 **Practical Performance Testing**



What is Performance Testing

Performance Testing

1

Time Behavior

2

**Resource
Utilization**

3

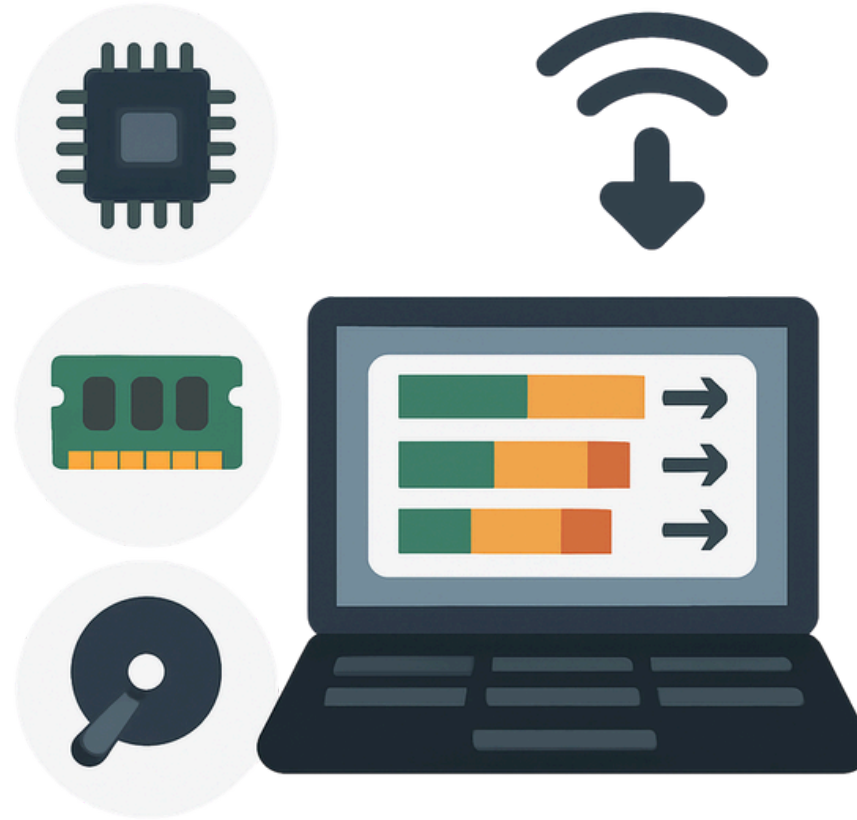
Capacity

Time Behavior



- Measures how long it takes the system to respond to request or perform an action
- **Example:** How many milliseconds it takes to load a webpage or complete a transaction.

Resource Utilization



- Monitors how much of the system's resources are consumed during operation [CPU Usage – Memory Usage – Network Bandwidth]
- **Example:** Streaming functionality causes network usage to exceed 800 Mbps

Capacity



- Identifies the maximum number of users or transactions the system can handle before performance degrades
- **Example:** A system might support up to 500 users before response time spikes or errors occur.

Performance Testing

- Performance testing is an umbrella term including any kind of testing focused on performance (**responsiveness**) of the system or component under different volumes of load.



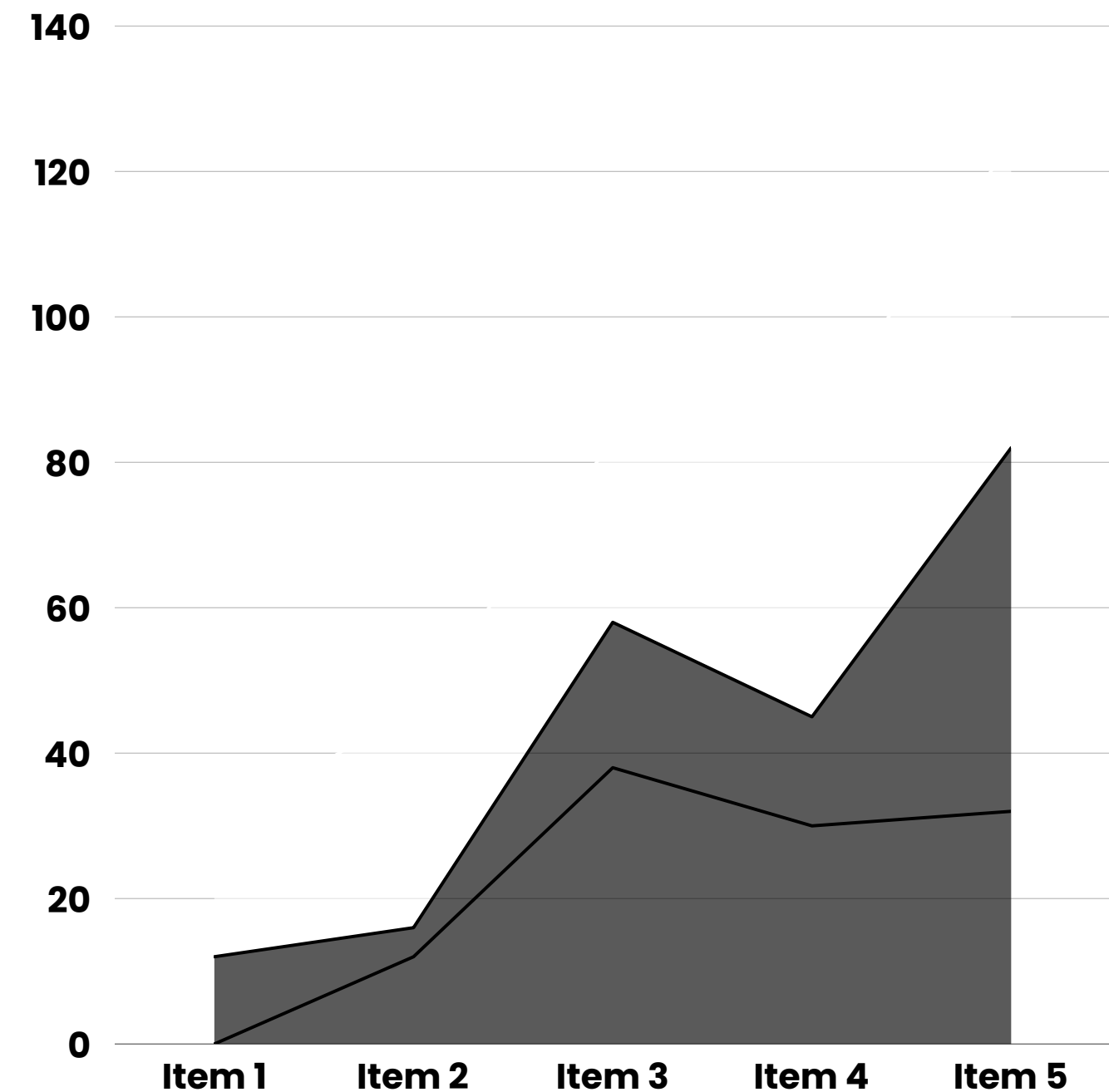
Load Testing

- Load testing focuses on the ability of a system to handle increasing levels of anticipated realistic loads resulting from transaction requests generated by controlled numbers of concurrent users or processes.



Stress Testing

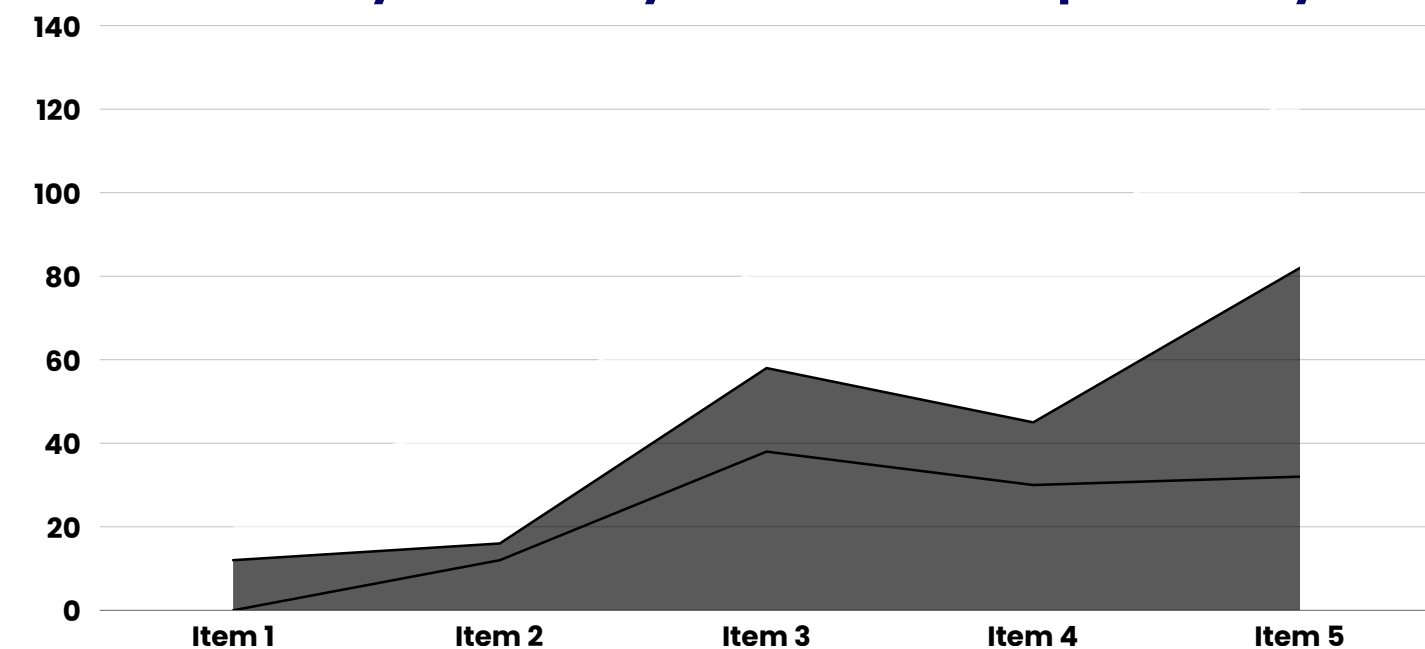
- **Focus is on the ability of a system to perform under extreme or unexpected load conditions, often beyond its normal operational capacity**



Load Testing VS Stress Testing



- Load tests simulate normal traffic, stress tests simulate extreme traffic.
- Load tests might crash your test environment, stress tests must break your test environment.
- Load tests determine if you can handle an expected traffic volume, stress tests determine the upper limits of your system capacity.



Scalability Testing

- Scalability testing focuses on the ability of a system to meet future efficiency requirements which may be beyond those currently required.



Capacity Testing

- Capacity testing determines how many users and/or transactions a given system will support and still meet the stated performance objectives



Endurance Testing

- Endurance testing focuses on the stability of the system over a time frame specific to the system's operational context.



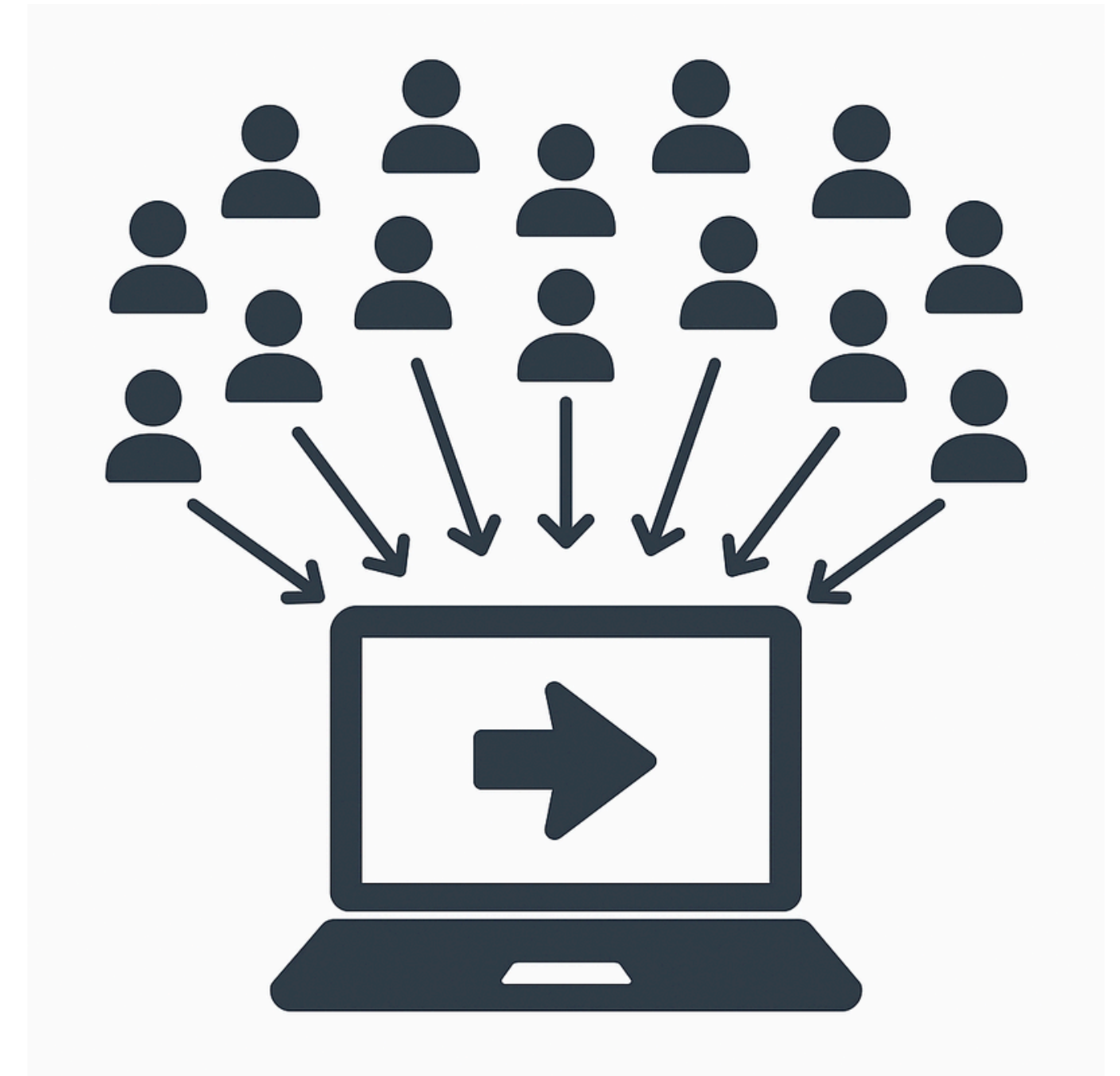
Concurrency Testing

- Concurrency testing focuses on the impact of situations where specific actions occur simultaneously (**e.g., when large numbers of users log in at the same time**)

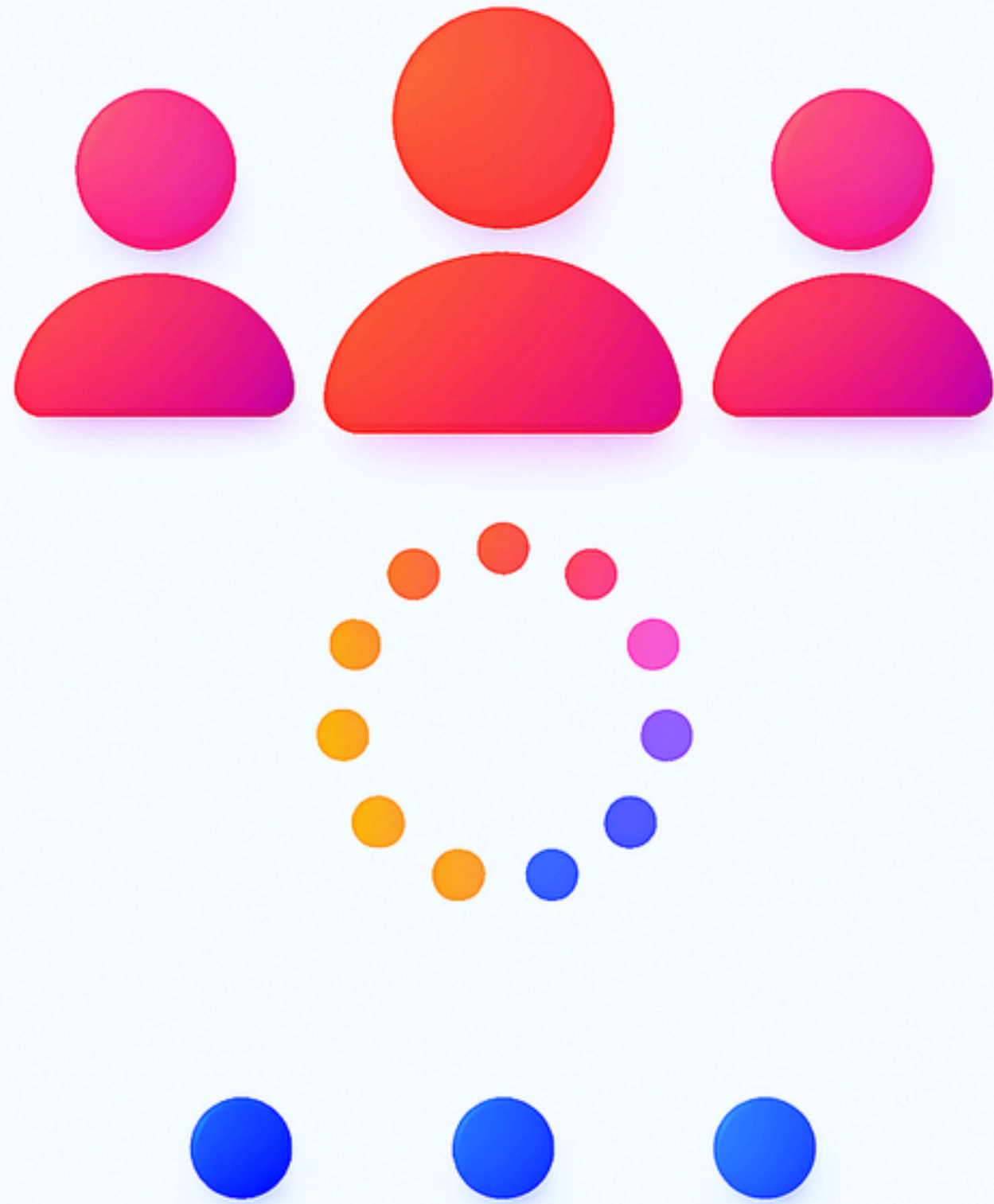


Load Generation

Load generation is the process of simulating multiple users or system requests on an application or system under test to evaluate its behavior, stability, and performance under expected or peak usage conditions



User Interface



Example:

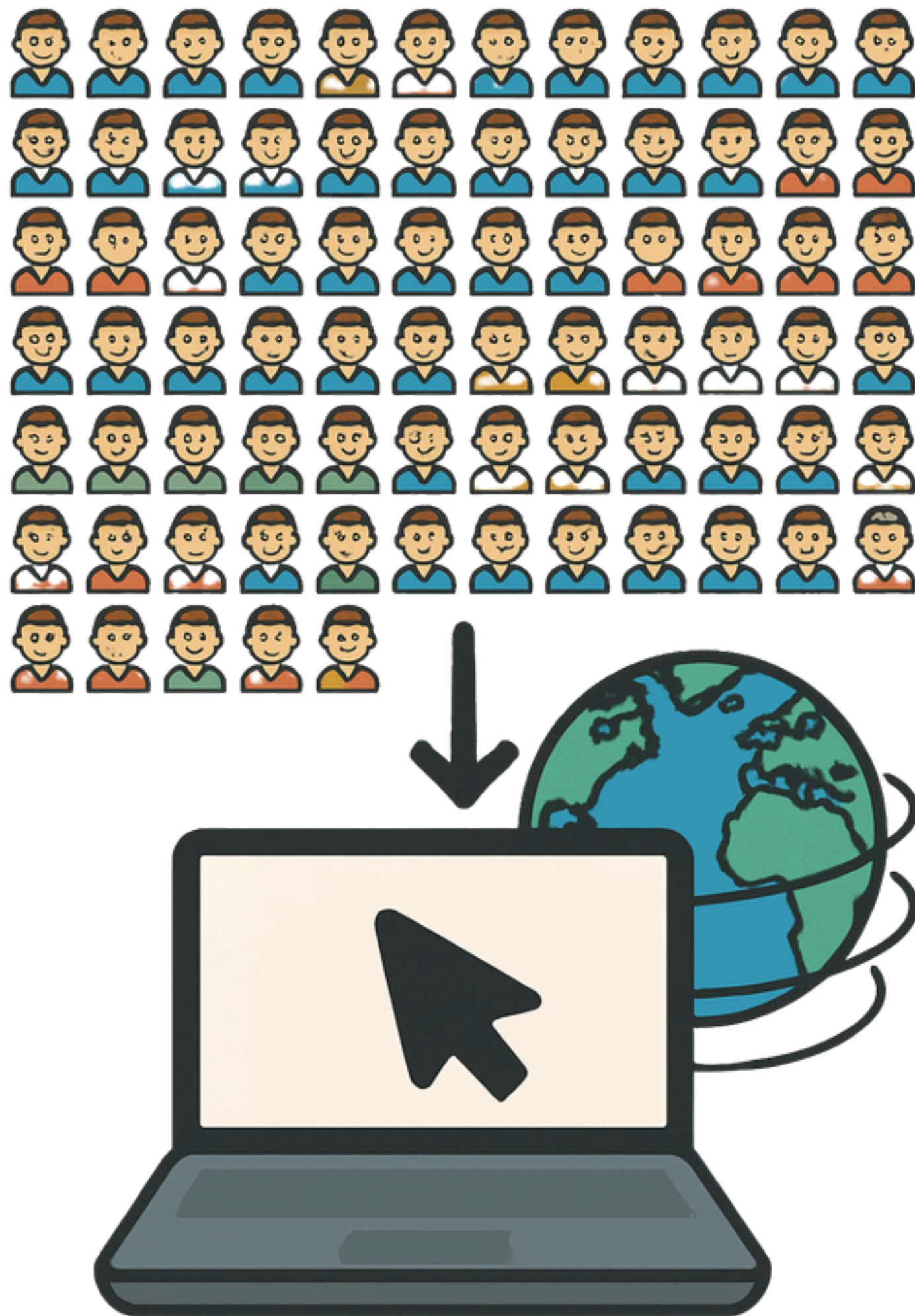
You use a tool like Selenium to simulate multiple users logging into a web application, clicking buttons, filling forms, and navigating through pages — just as a real user would do in a browser.

Use Case: Ideal for testing the front-end responsiveness of your app when a small number of users are performing complex interactions.

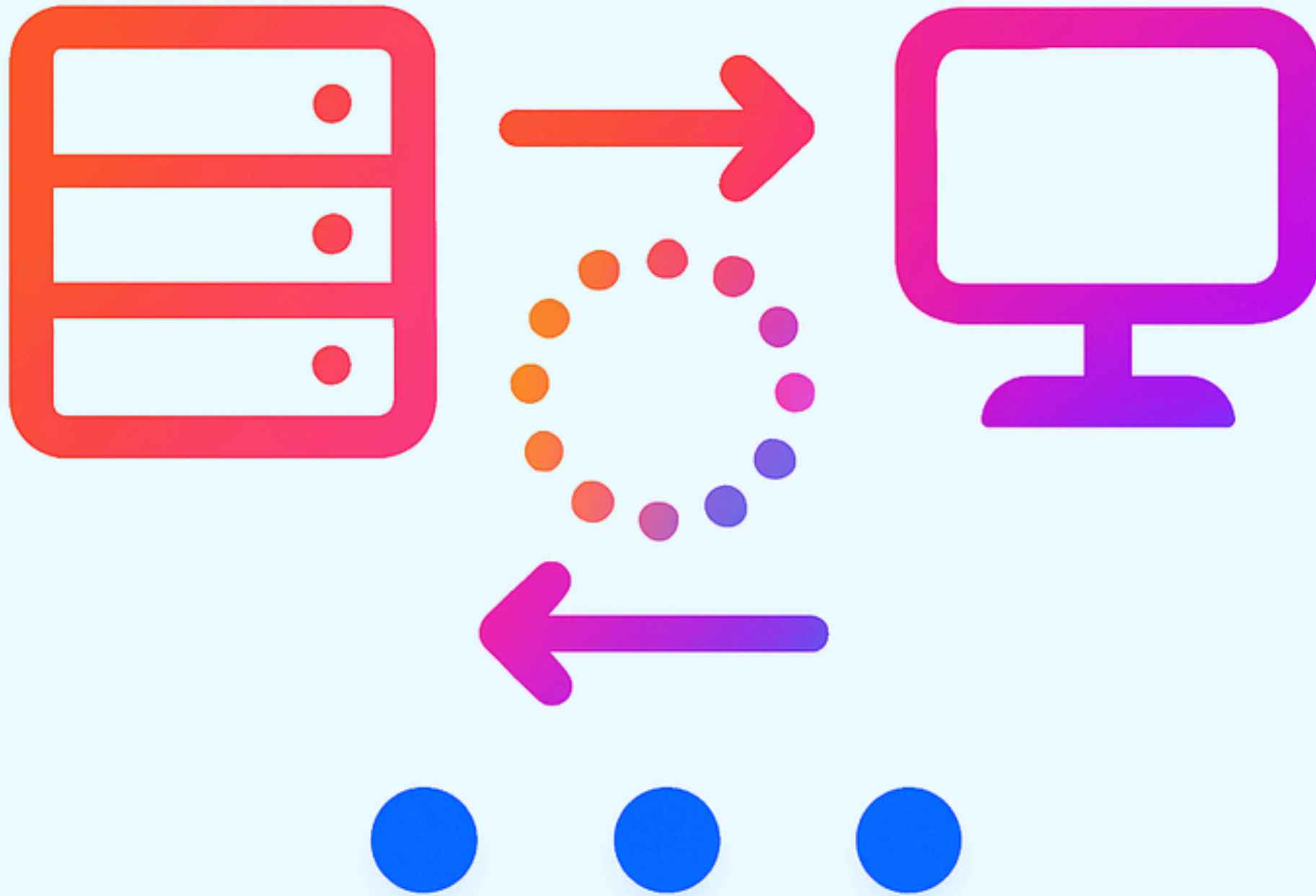
Crowds

Example: Hiring or coordinating 100 real users from different geographic locations to access the system simultaneously.

Use Case: Testing a new video conferencing platform (like Zoom or Google Meet) by having all 100 users join the same meeting at once, to evaluate the system's scalability, audio/video stability, and server load handling.

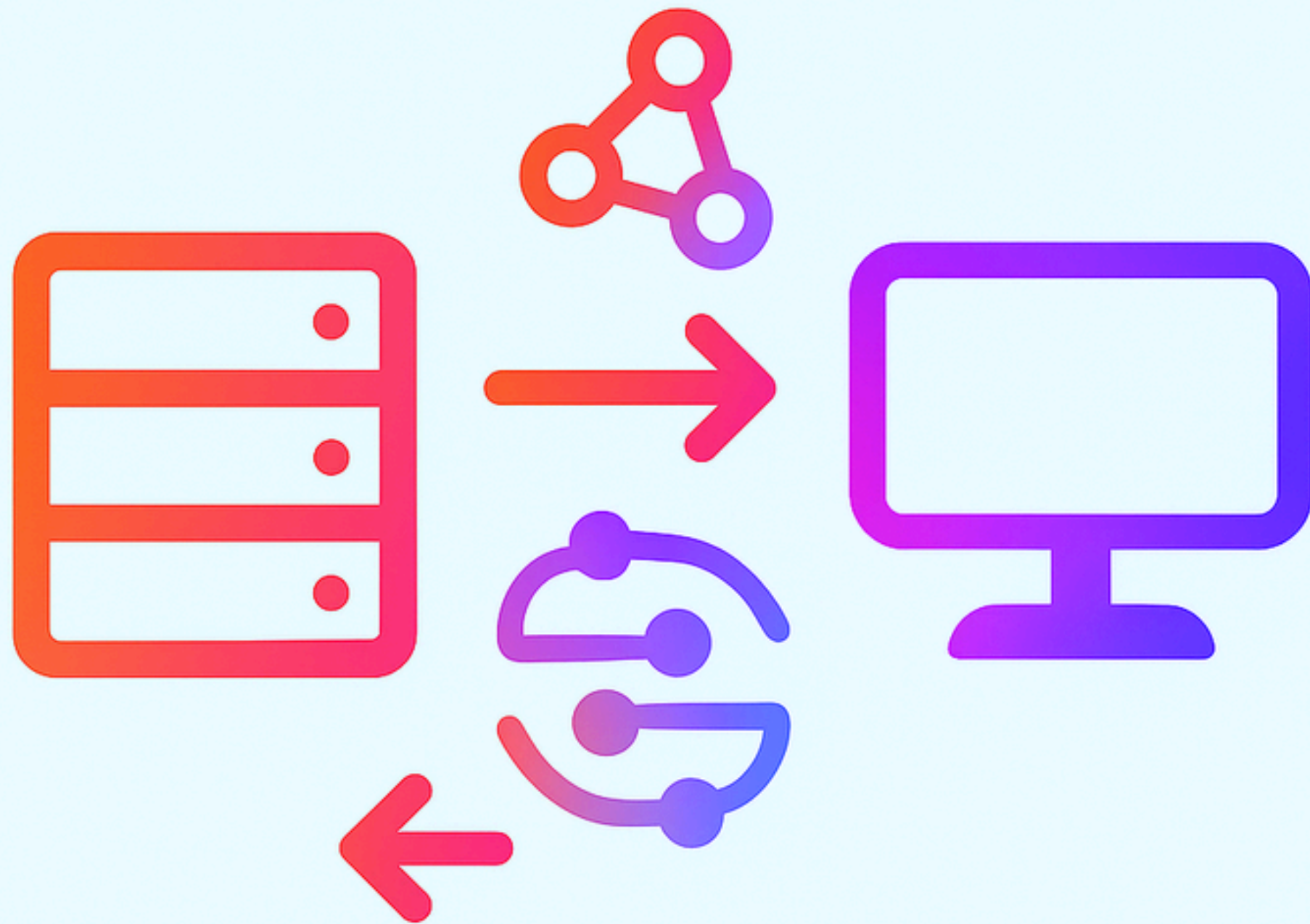


APIs



Example: You use tools like JMeter, Postman Runner, or k6 to send high volumes of HTTP requests

Use Case: Running Collection Runner in Postman to mimic a group of HTTP requests multiple times. Less sensitive to UI Changes



COMMUNICATION PROTOCOLS

Communication protocols

Example: Using JMeter, LoadRunner, or Gatling to generate load using lower-level protocols like **TCP**, **MQTT**, **WebSockets**, **FTP**, or **SMTP**

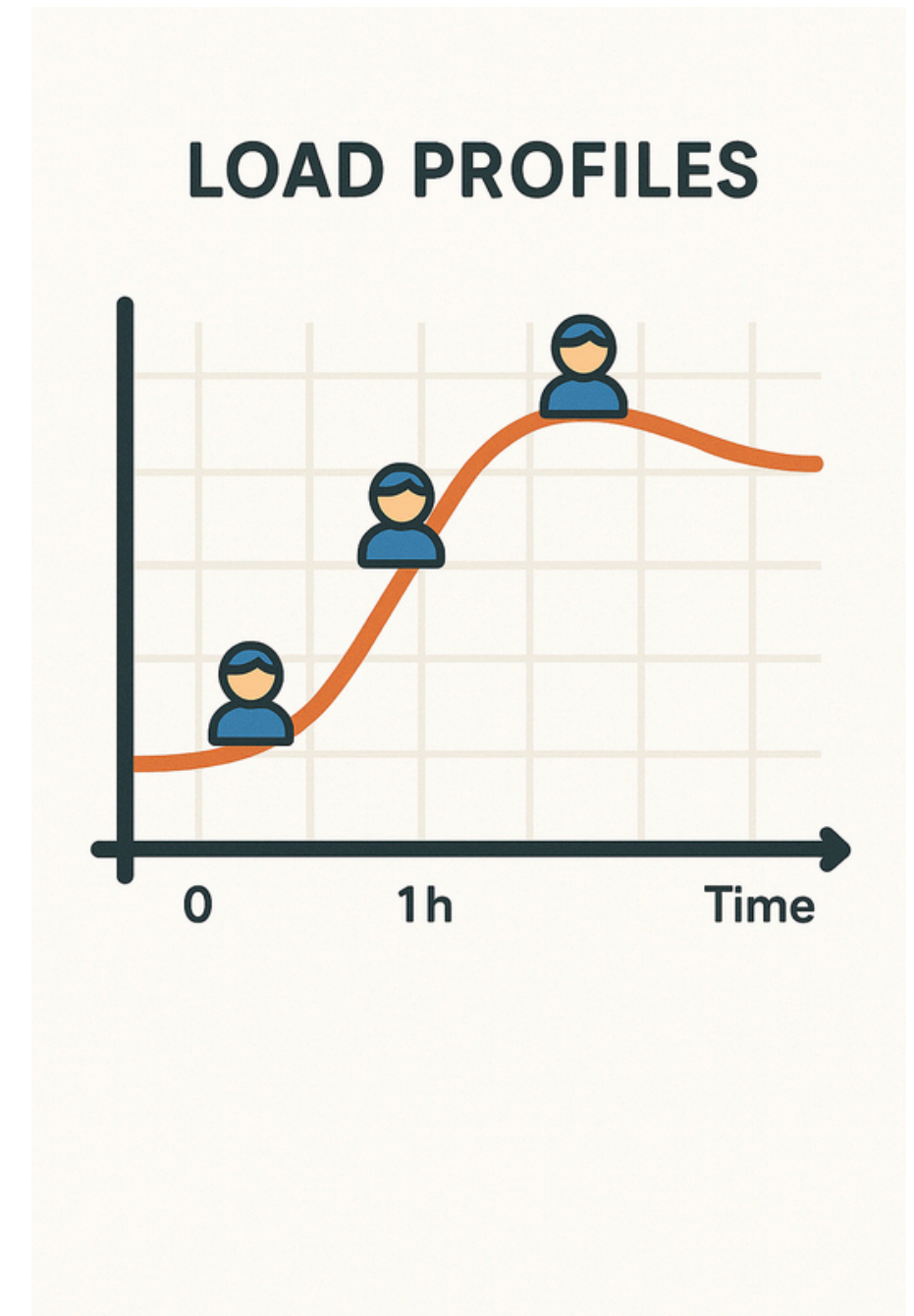
Use Case: Required when you want to test custom-built services or devices, such as simulating thousands of **IoT** devices sending sensor data using **MQTT**



Creating Load Profiles

Creating Load Profiles

- A **load profile** specifies the activity which a component or system being tested may experience in production. It consists of a designated number of instances that will perform the actions of predefined **operational profiles** over a specified time period.

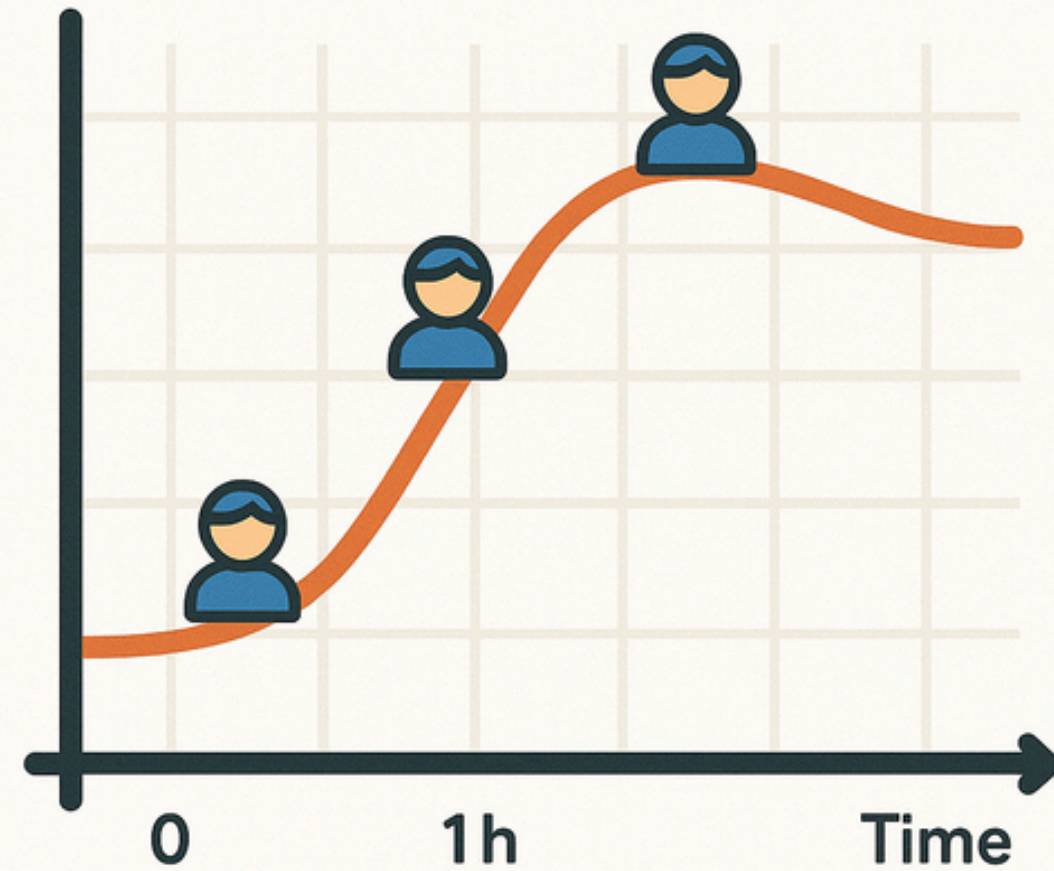


Creating Load Profiles

To create a correct load profile, you need to understand the following:

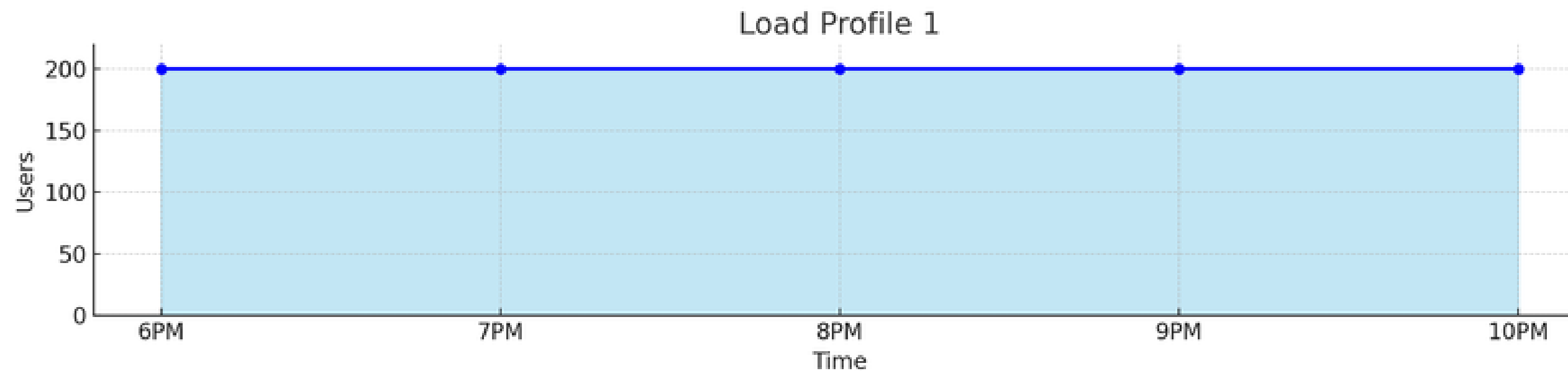
- Performance Testing Objectives
- Operational Profiles which accurately represent individual usage patterns
- The quantity and time distribution with which the operational profiles are to be executed such that the **SUT** experiences the desired load. Examples are:
 - **Ramp-ups:** Steadily increasing load (**e.g., add one virtual user per minute**)
 - **Ramp-downs:** Steadily decreasing load
 - **Steps:** instantaneous changes in load (**e.g., add 100 virtual users every five minutes**)
 - **Predefined distributions:** (**e.g., volume mimics daily or seasonal business cycles**)

LOAD PROFILES



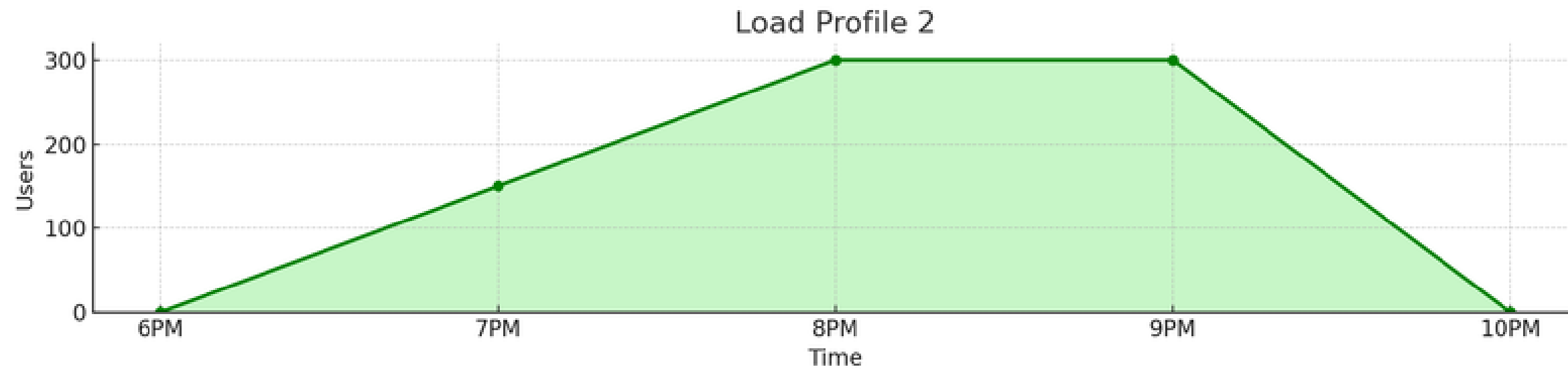
Load Profiles Examples

Load Profile 1 (Background Load)



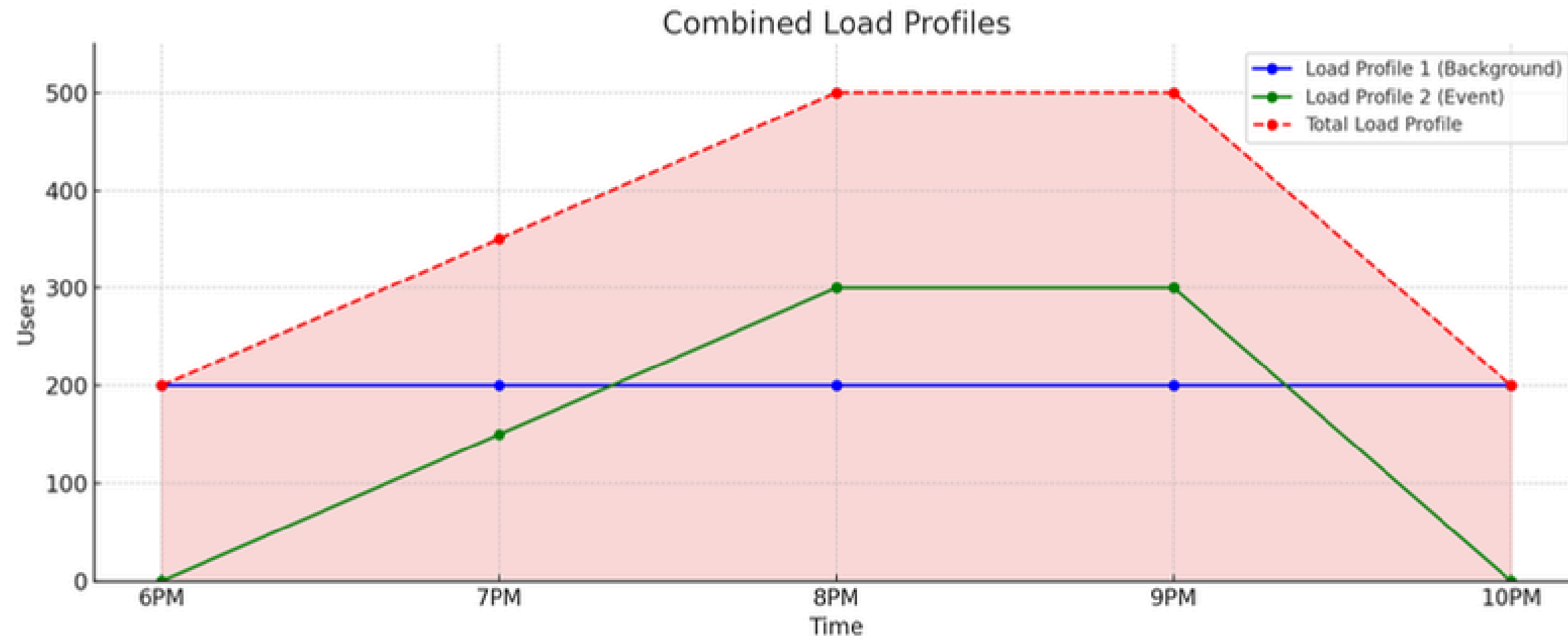
- A load profile is shown which consists of a steady load of **200 virtual users** from 6 PM to 10 PM. These users perform the actions defined by **Operation Profile 1** throughout the entire test duration. This pattern typically represents a **background load**, simulating continuous user activity in the system.

Load Profile 2 (Time-limited Load)



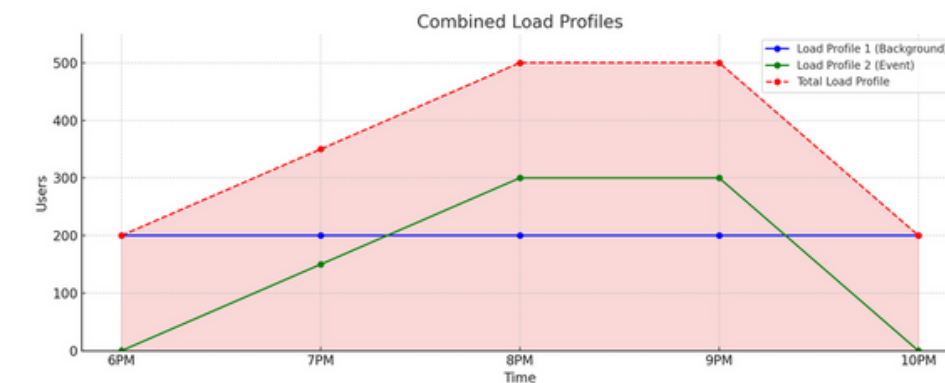
- **Load Profile 2** simulates a **time-limited** event load. The profile starts with 0 users at 6 PM, followed by a gradual **ramp-up** reaching a peak of 300 users by 8 PM. The load remains steady at 300 users between 8 PM and 9 PM, then **ramps down** back to 0 users by 10 PM.

Load Profile 3 (Combined load)

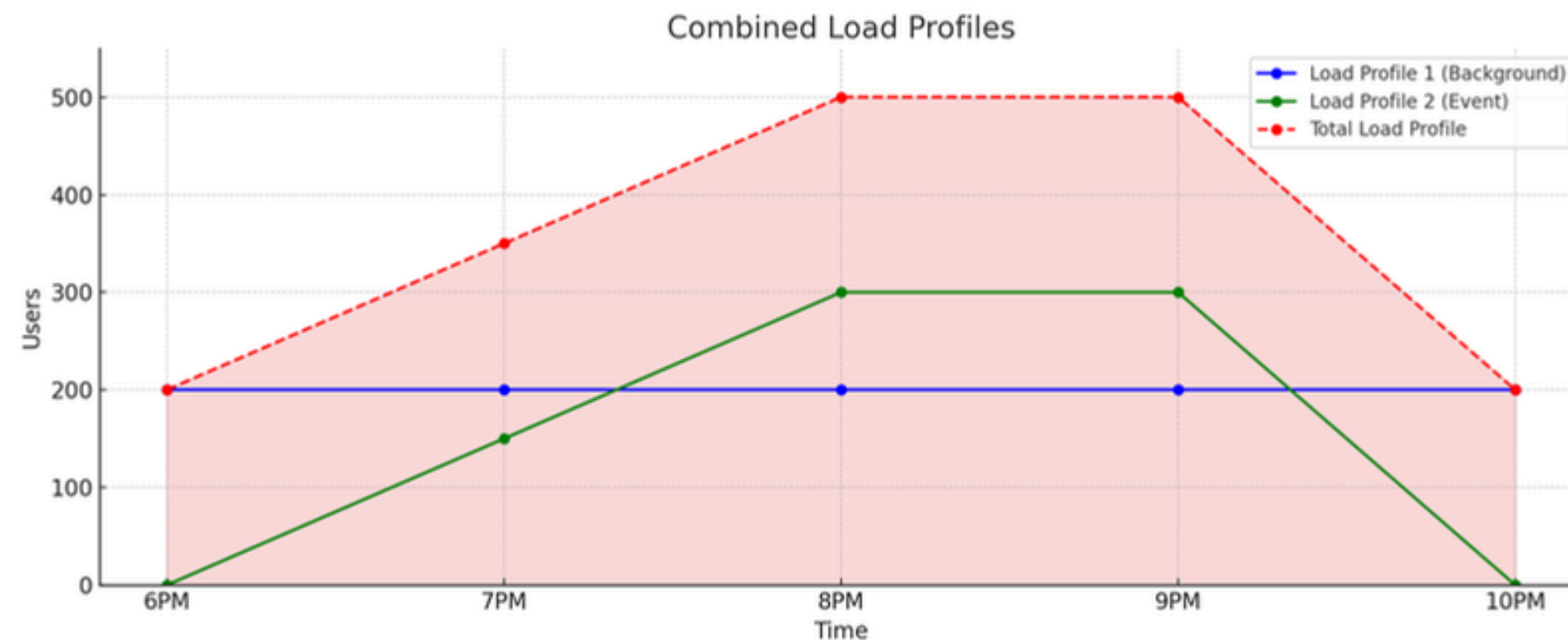
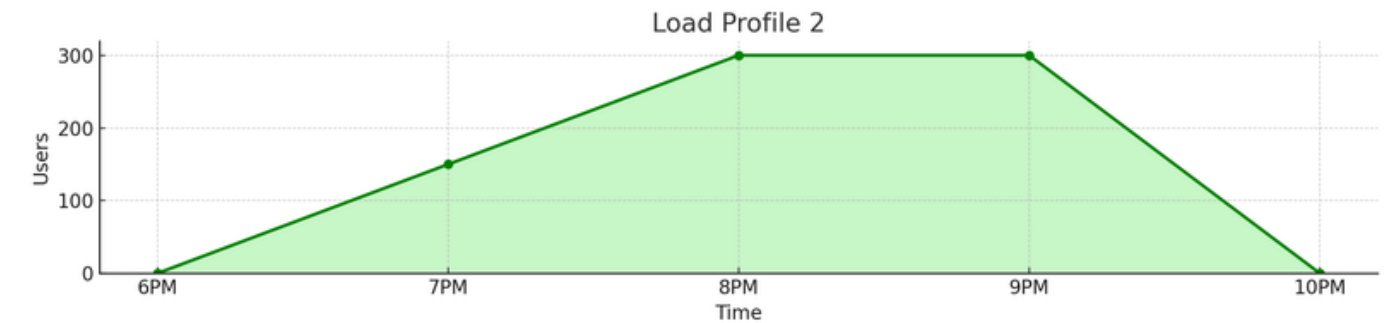
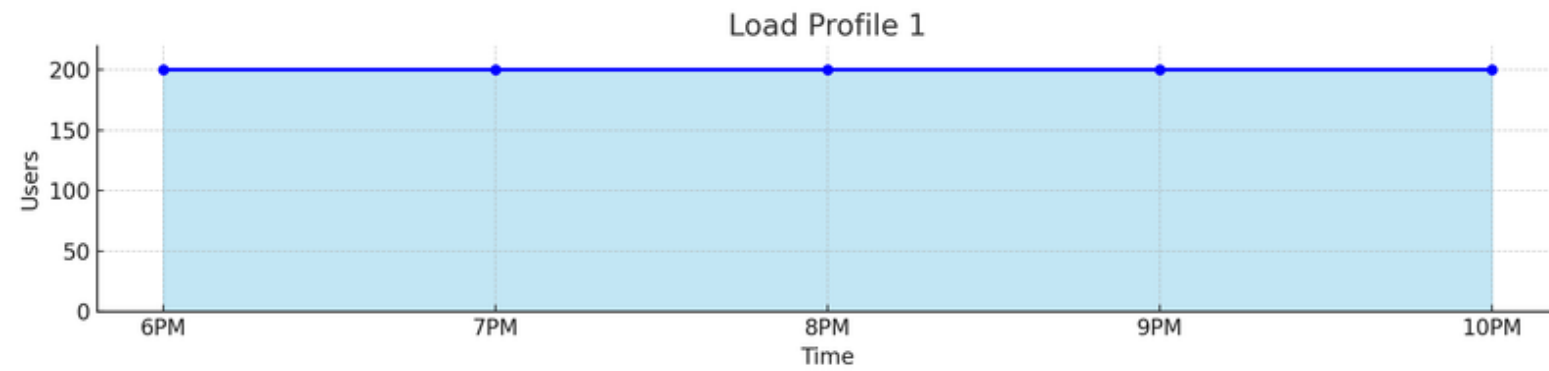


Load Profile 3 (Combined load)

- **Load Profile 3** represents the total system load by combining a constant background load (**Profile 1**) with a time-limited event load (**Profile 2**).
- The background load maintains a steady level of **200 users** throughout the entire period from **6 PM to 10 PM**. Meanwhile, the event load introduces a gradual increase starting at 6 PM, reaching an additional 300 users at 8 PM, staying at that peak until 9 PM, and then dropping back to 0 by 10 PM.
- As a result, the total system load **peaks at 500** users during the event window (8 PM–9 PM), making this profile ideal for testing how the system behaves under both steady background activity and temporary spikes in user demand.



Creating Load Profiles Example





What are example test cases that can be used in Endurance Testing, Load Testing, and Stress Testing for this system?

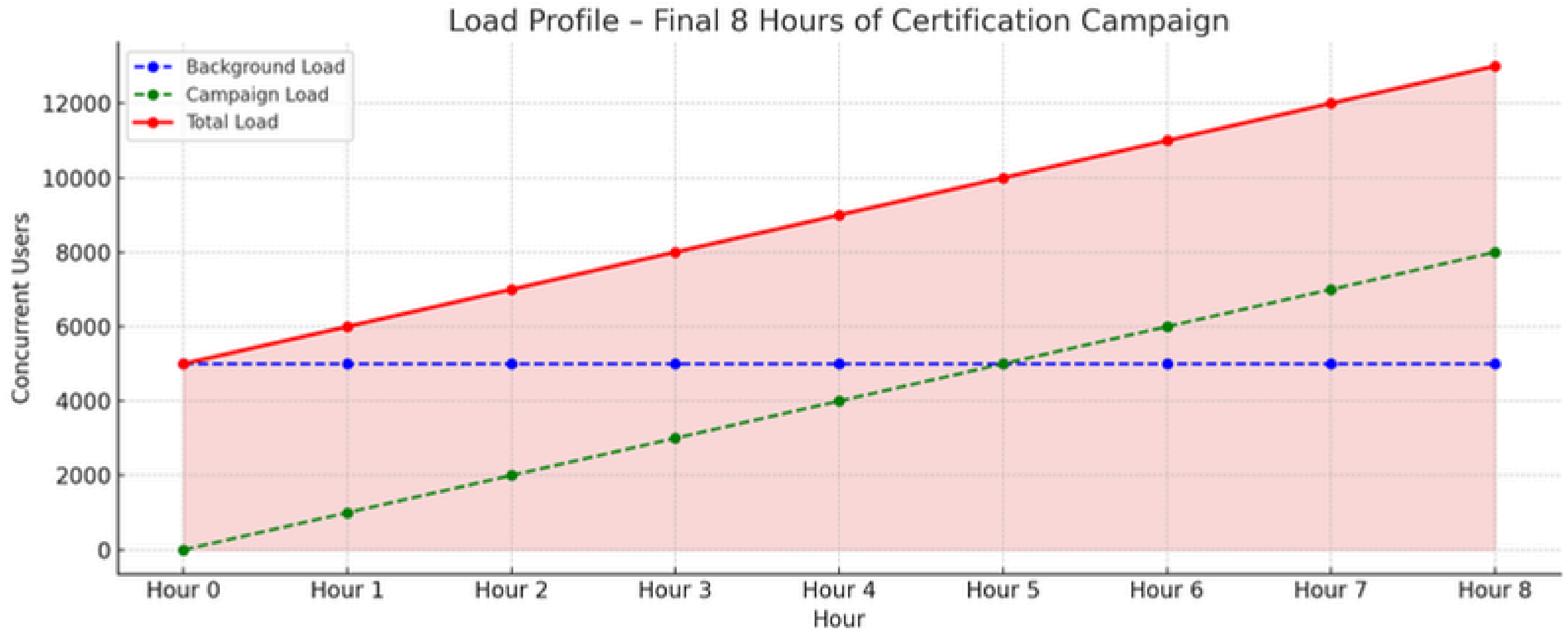
Endurance, Load, & Stress Testing Test Cases

Testing Type	Test Case
Endurance Testing	Simulate 200 users continuously using the system (login, search, logout) for 8+ hours to detect memory leaks or performance degradation
Load Testing	Simulate 500 users performing typical operations like product search, add to cart, and checkout over 1 hour to evaluate system behavior under expected load.
Stress Testing	Gradually increase users from 0 to more than 500 users until the system fails, to identify breaking points and recovery behavior.

Load Profile Example #2

- An online course platform is running a global certification campaign.
- Users are expected to complete the course and claim their certificate during the final 8 hours before the deadline. There's a constant background load of 5,000 users unrelated to the campaign. Campaign-related traffic is expected to increase steadily each hour, as follows:
 - Hour 1: 1,000 users
 - Hour 2: 2,000 users
 -
 -
 - Hour 8: 8,000 users
- Each session includes video viewing, quizzes, and certificate generation.
- **Design a load profile that shows how campaign-related users combine with background users. What is the peak number of concurrent users expected during the campaign window? How can this data be used in performance testing?**

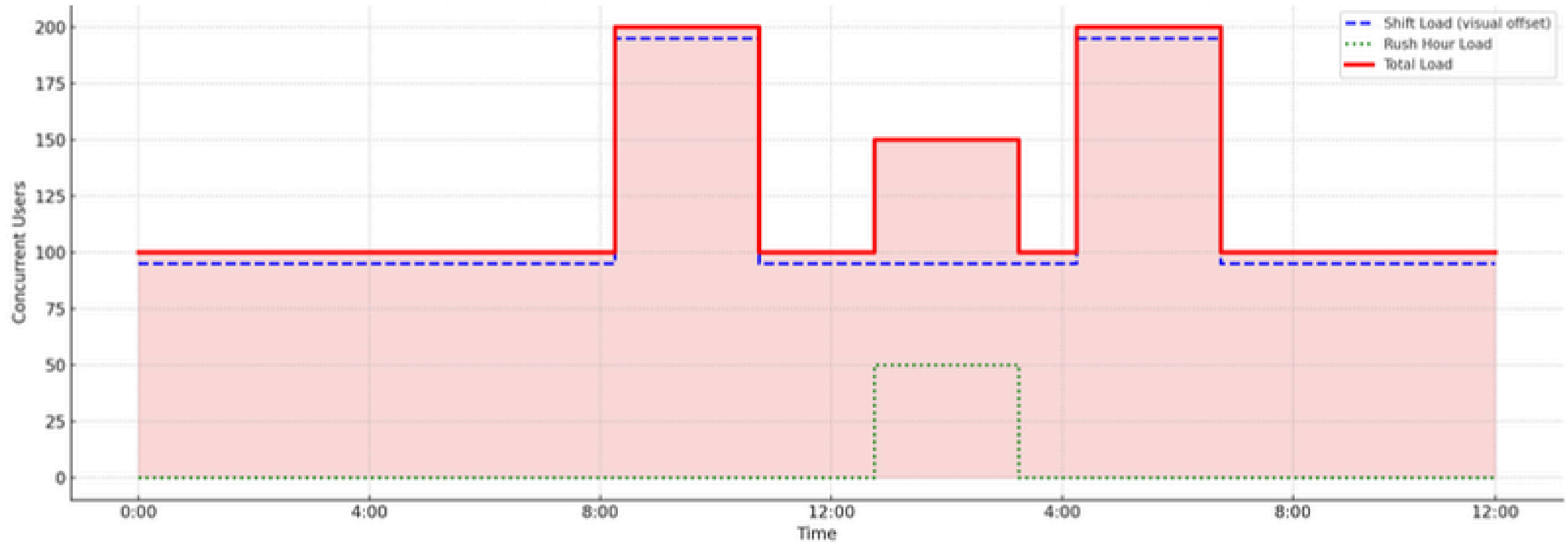
Example #2 Solution

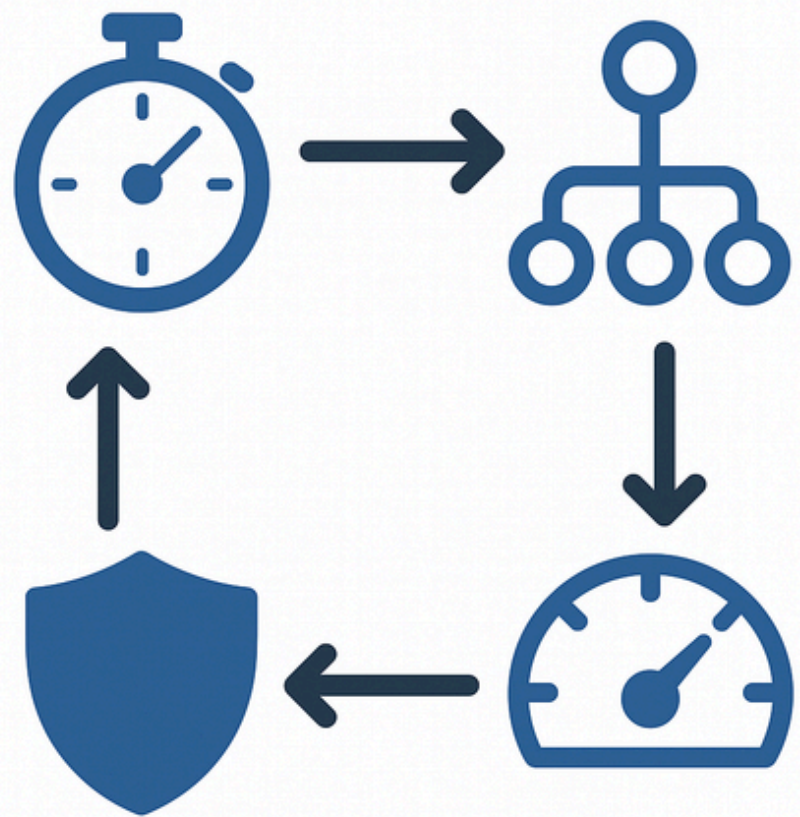


Load Profile Example #3

- You are testing the performance of a system used by traffic officers to log incidents, check vehicle data, and update road conditions.
- The system is accessed by officers working across three shifts:
 - Shift 1: 12:00 AM – 10:00 AM
 - Shift 2: 08:00 AM – 6:00 PM
 - Shift 3: 04:00 PM – 12:00 AM
- Each shift includes 100 officers who remain logged in during their entire shift.
- Additionally, from 1:00 PM to 3:00 PM, there are 50 extra officers on duty due to rush hour, unrelated to the regular shifts.
- Task:
 - Design a load profile over 24 hours showing expected concurrent users.
 - At what times does the system experience peak load?
 - What is the highest number of concurrent users?

Example #3 Solution



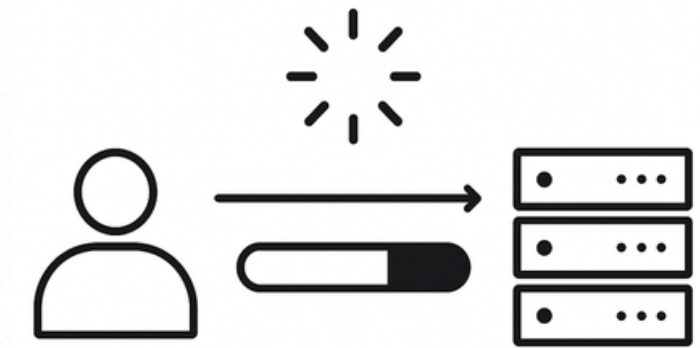
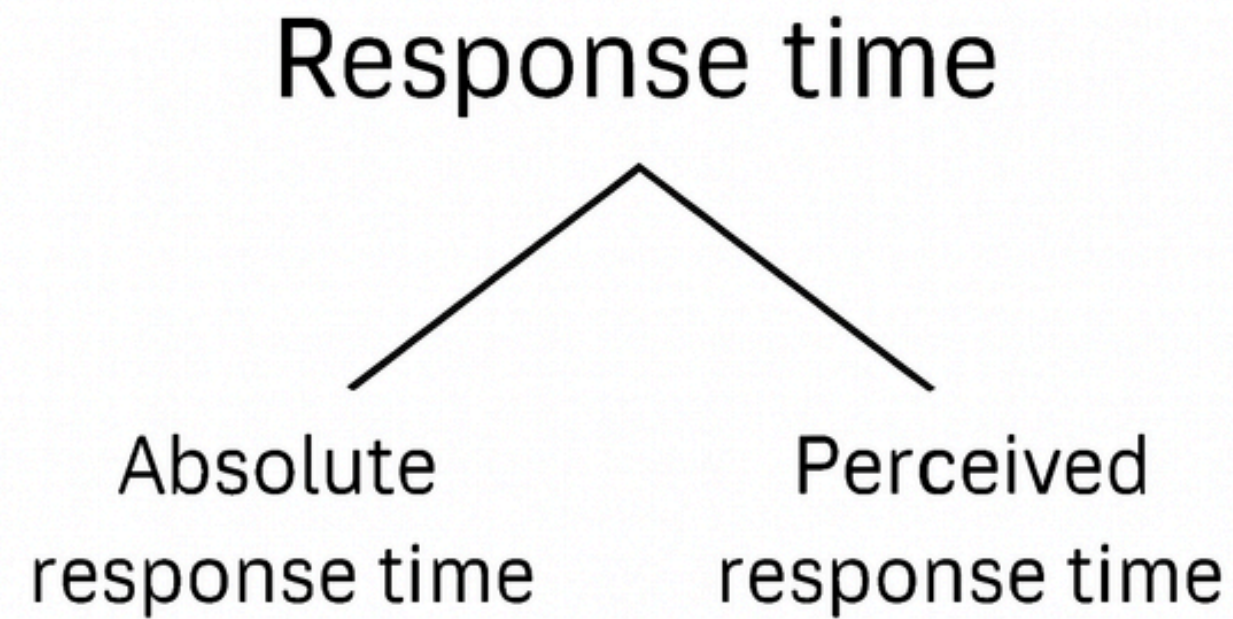
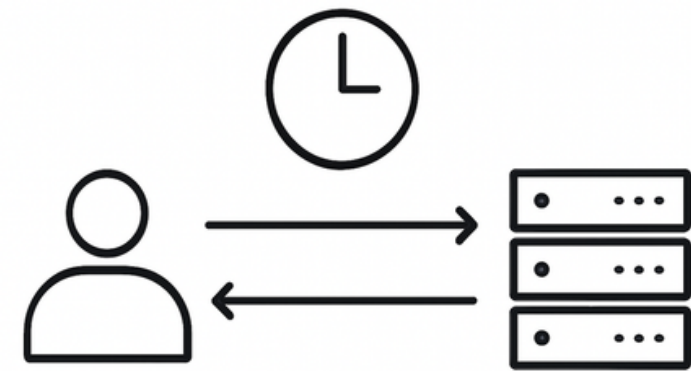


**Response time,
Throughput, Utilization, &
Robustness**

Response Time



Time taken by the application to respond to user's
request



- **Definition:** The total amount of time it takes for a system to process a request and return a response.
- **Measurement:** Usually measured from the time a request hits the server until a complete response is returned.
- **Objective:** It's a technical measurement, often captured by tools or logs (e.g., backend response time).
- **Example:** The server processes a search query and returns the result in 1.2 seconds.

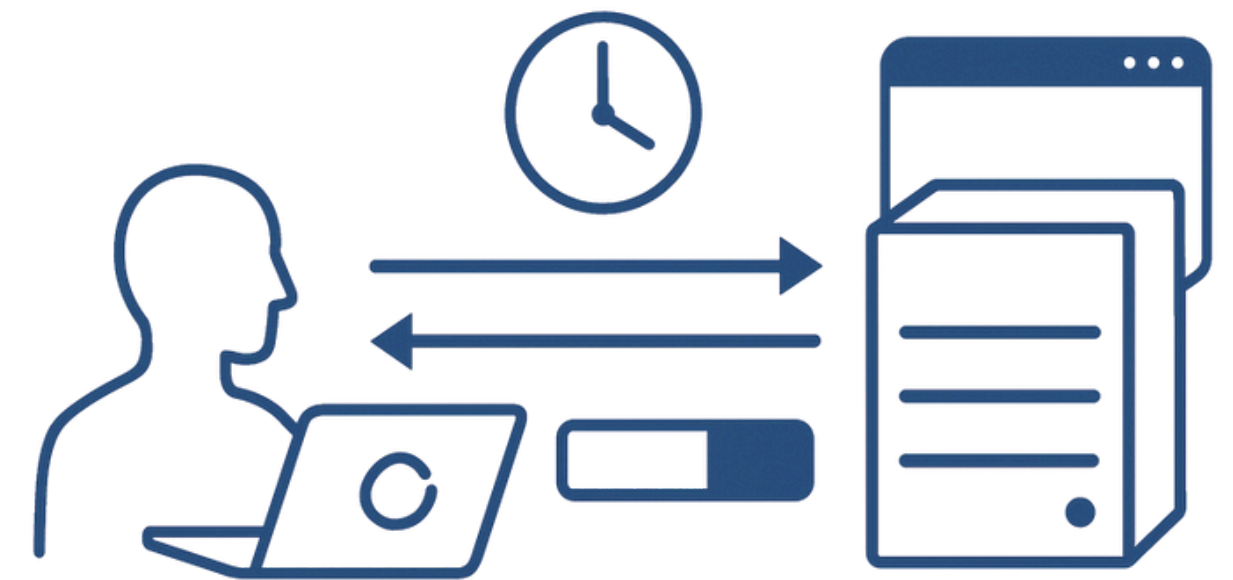
- **Definition:** The time a user feels it takes for the system to respond.
- **Measurement:** Includes not just processing time but also things like:
 - Network latency
 - UI rendering delays
 - Loading animations
 - Client-side processing
- **Subjective:** It's a user-focused metric — how fast the system “feels.”
- **Example:** The system technically responds in 1.2 seconds, but due to spinner animations or delayed display, the user feels it took 2 seconds.

Aspect	Absolute Response Time	Perceived Response Time
Perspective	System/server	User
Measured By	Logs, monitoring tools	User feedback, UX studies
Includes	Backend processing	Backend + frontend + UX effects
Importance	Ensures system performs efficiently	Ensures users feel the system is fast

Rendering Time

- Time taken by the browser to parse and render the response received from the server.

Network Latency



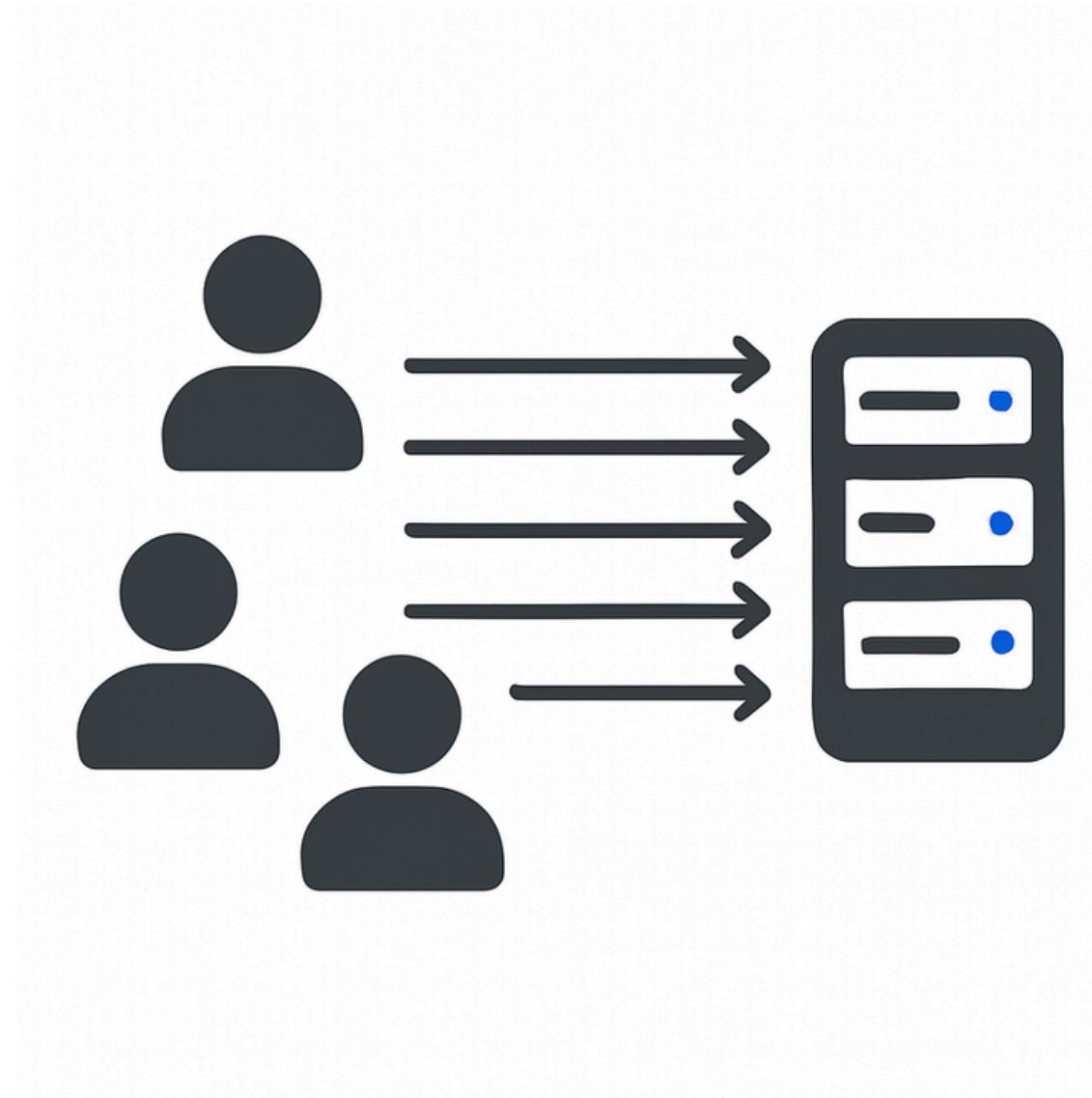
- Time taken by a data packet to be captured, transmitted, and processed through multiple devices, then received at its destination.

Perceived Response Time



Server Response Time + Rendering Time + Network Latency

Throughput



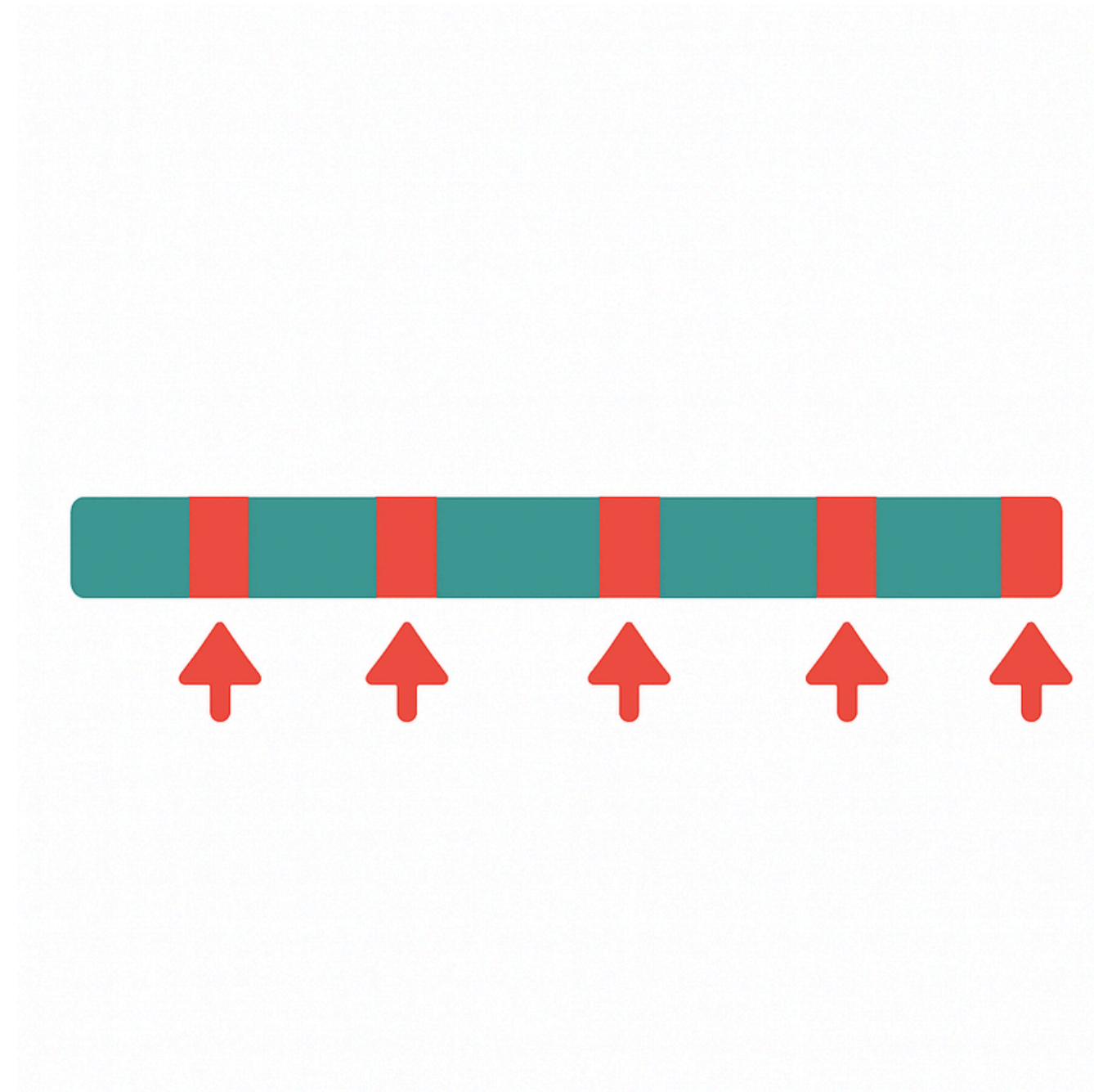
Number of transactions per unit time Measured in transactions/second or bandwidth (**bytes/second**)

Utilization



Ratio of throughput to maximum capacity. It is not desirable to operate above **80%** utilization

Robustness



How well the application detects and handles various errors and exceptions. Measured using **MTBF (Mean Time Between Failures)**



Performance Test Environment

Performance Test Environment



A dedicated setup where performance testing is executed to simulate real-world conditions.

Performance Test Environment



- Performance Test Environment should **mimic** real **production** environment
- Performance Test Environment should be **isolated**



**Why do we need a
separate test environment?**

Why do we need a separate test environment?



- The system might crash
- Application response time is degraded
- Security holes might be created because of the use of test accounts
- Production database will include performance test input and output data
- Application log files and system log files may be filled up
- Analytics will be influenced

STATISTICS CRASH COURSE



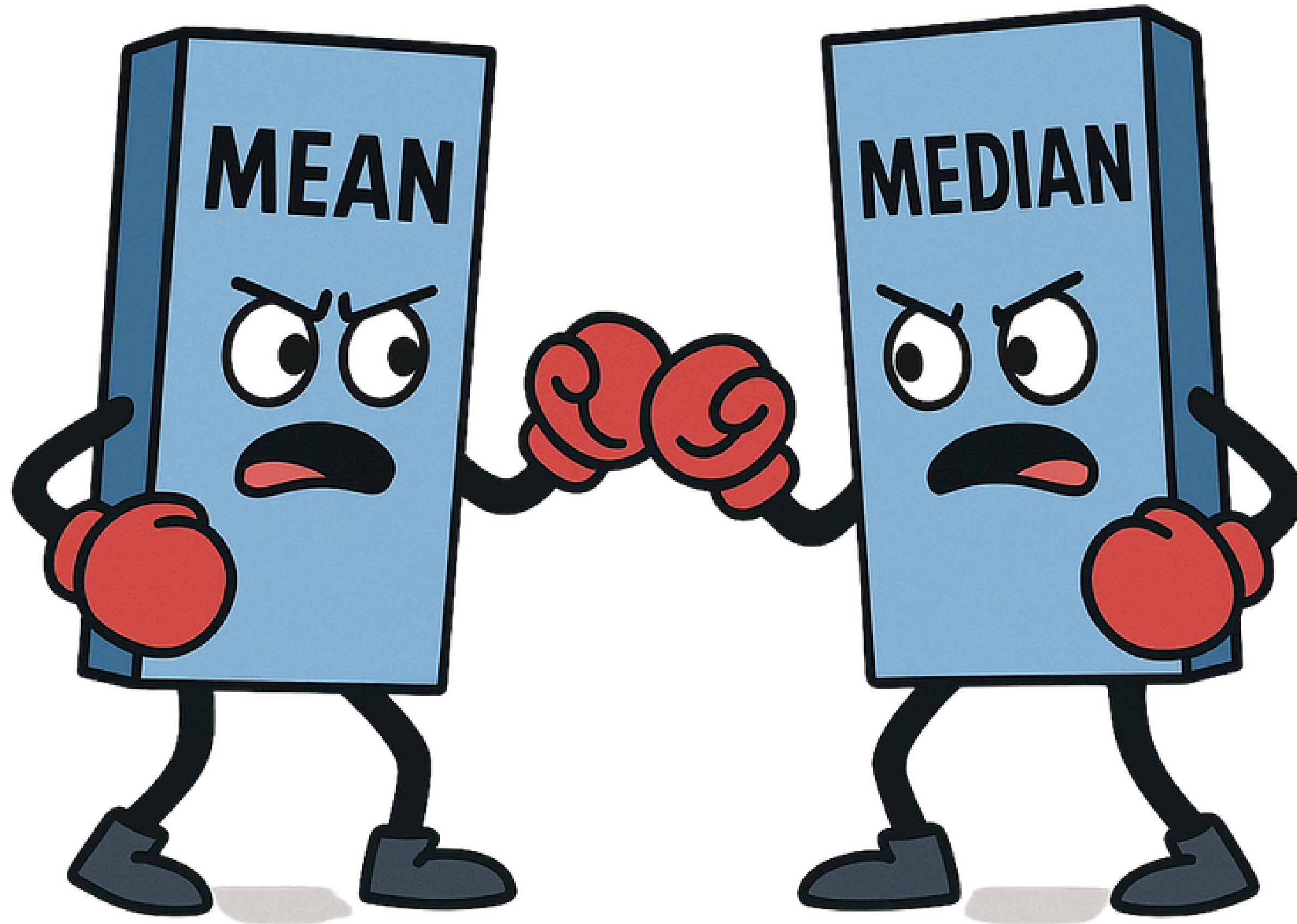
Average (Arithmetic Mean)

- **Definition:** Sum of all values divided by the number of values.
- **Use Case:** Gives a general idea of the “typical” value.
- **Limitation:** Sensitive to outliers (e.g., one slow response can skew it).
- **Example:**
 - $(1 + 2 + 3 + 4 + 5) / 5 = 3$ [**Descriptive**]
 - $(1 + 2 + 3 + 4 + 1000) / 5 = 202$ [**Misleading**]

Median

- **Definition:** The middle value when all values are sorted.
- **Use Case:** Good for understanding the “typical” user experience ignoring outliers.
- **Limitation:**
 - It gives a sense of the “typical” user experience, but not the **edge cases**.
 - You might miss performance regressions that affect a **minority** of users which are often the most critical (e.g., during peak usage).
- **Example:**
 - $[1, 2, \mathbf{3}, 4, 100] \rightarrow \text{Median} = 3$ [**Descriptive**]
 - $[1, 2, 3, \mathbf{4}, 1000, 1000, 1000] \rightarrow \text{Median} = 4$ [**Misleading**]

Which one should you use?



[100, 100, 100, 100, 100, 200, 300, 400, 900, 5000]

- | | |
|--|--|
| <ul style="list-style-type: none">• Mean = 740 ms• Makes it sound like performance is bad for everyone, but 8 out of 10 users had ≤ 400 ms! • Alerts you that there's an outlier pulling up the average  | <ul style="list-style-type: none">• Median = 150 ms• makes it sound like the system is fast for everyone but one user waited 5 seconds! • shows that most users had a good experience  |
|--|--|

“The typical user is fine, but there’s a serious performance issue affecting a minority.”



**Is there another metric
that can be useful?**

Percentiles

A **percentile** is a value below which a given percentage of observations fall



- For example, the 90th percentile means:
 - 90% of the values are less than or equal to this value
- What about 50% percentile, what does it mean?

Let's have an example

[100, 102, 101, 103, 102, 100, 101, 102, 101, 800]

Sorted: [100, 100, 101, 101, 101, 102, 102, 102, 103, 800]

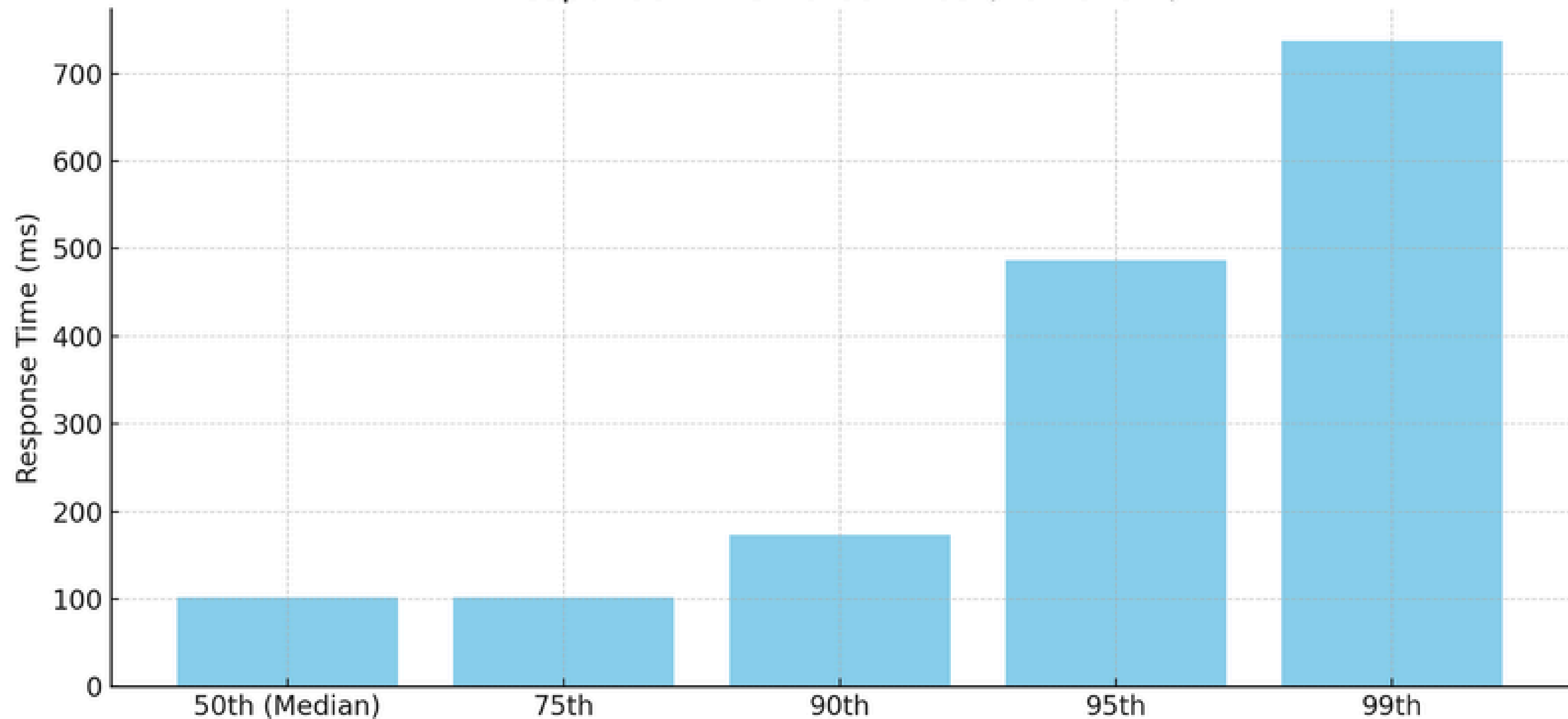
Percentile	Position	Value (ms)	Meaning
50 th	Middle	101	50% of users had ≤ 101 ms (same as median)
75 th	7.5 th	≈ 102	75% of users had ≤ 102 ms
90 th	9 th	103	90% of users had ≤ 103 ms
95 th	9.5 th	≈ (103 + 800)/2 = 451.5	95% had ≤ 451.5 ms
99 th	9.9 th	≈ 800	Almost everyone had faster responses, except one extreme case

Percentiles

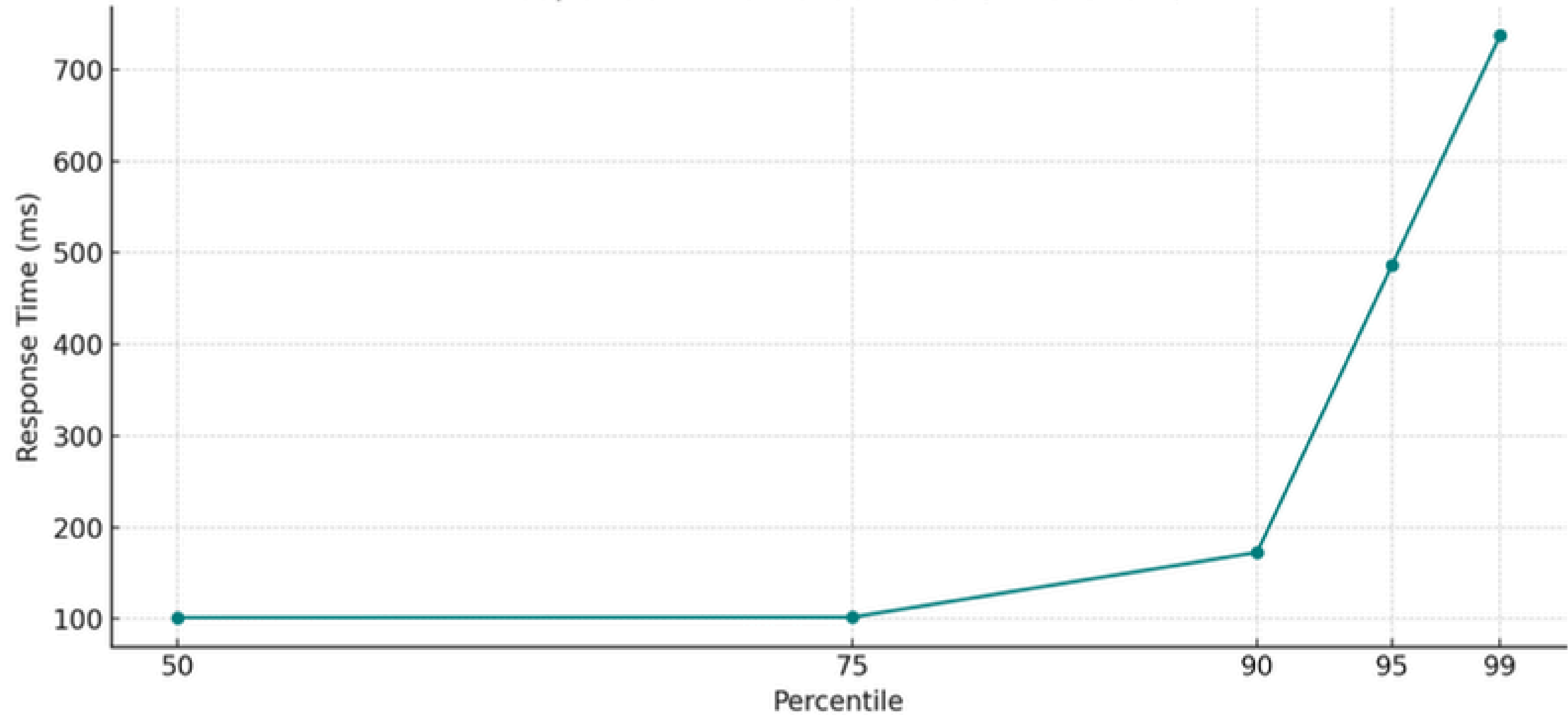


- **Low percentiles (50th–75th)** → show how most users perform.
- **High percentiles (90th–99th)** → expose outliers and slowest experiences.

Response Time Percentiles (Bar Chart)



Response Time Percentiles (Line Chart)



Standard Deviation



Standard Deviation (SD) is a measure of how spread out the values in a dataset are around the mean (average)

Standard Deviation Examples

Values	Standard Deviation
100, 100, 100, 100, 100 ms	0 ms
98, 100, 101, 99, 102 ms	1.41 ms
25, 75, 100, 125, 175 ms	50 ms
25, 25, 50, 100, 300 ms	103.68 ms

Low SD → Consistent performance ✓

High SD → Performance spikes ✗

performance Testing using



The Complete Software Testing Bootcamp

What is JMeter

- Apache JMeter is an open-source, Java-based software application designed for load testing and performance measurement.
- It is not limited to: **HTTP** — also supports **FTP**, **SOAP**, **REST** APIs, **JDBC** (Databases), **JMS**, **SMTP**, etc.

⚙️ Why Use **JMeter**?

- Simulates multiple users to test how a system behaves under load
- Supports distributed testing (multiple machines)
- Rich plugin ecosystem for extended functionality
- Can generate detailed reports (graphs, tables, summaries)



Installation Steps (Windows)

1. Install Java JDK

2. Download & Extract JMeter

Go to: https://jmeter.apache.org/download_jmeter.cgi

3. 🚀 Run JMeter

- Navigate to: apache-jmeter-x.x/bin
- Double-click jmeter.bat



Installation Steps (MacOS)

1. Install Java JDK

2. Install JMeter through Homebrew

brew install jmeter

3. 🚀 Run JMeter

- **JMeter**





Samplers





Listeners





Adding Blazemeter Plugin





Recording scripts using Blazemeter





Replaying recorded scripts





Advanced Performance Testing using JMeter

Created by : Tarek Roshdy



Advanced Performance Testing



Serial Execution of Thread Groups



User Defined Variables



Action After Sampler Error



Controllers - Simple Controller

CONTROLLERS

Jmeter Tutorial

SIMPLE CONTROLLER

Jmeter Tutorial



Loop Controller – Runtime Controller



Throughput Controller

- Once Only Controller**
- Interleave Controller**